Doc. no.:	P2511R1
Date:	2022-3-14
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy at miator dot net>

Beyond operator(): NTTP callables in type-erased call wrappers

Changes Since R0

- respond to offline comments
- simplify semantics
- provide wording for `std::function`

Introduction

Non-type template parameters (NTTP) can provide information for call wrappers to erase at compile-time, eliminating the need for binding objects at runtime when fulfilling a simple demand. This paper proposes tweaking type-erased call wrappers, such as `std::move_only_function`, with NTTP callable objects.

Here is an unfair Tony Table that quickly demonstrates a motivating use case:

C++11	`q.push(std::bind(&DB::connect, std::move(db), _1, _2, _3));`

C++14	`q.push([db{std::move(db)}] (auto &&... args) mutable`
	`{`
	`return db.connect(std::forward<decltype(args)>(args)...);`
	`});`

C++20	`q.push(std::bind_front(&DB::connect, std::move(db)));`
	
	`q.push([db{std::move(db)}] <class ...T>(T &&... args) mutable`
	`{`
	`return db.connect(std::forward<T>(args)...);`
	`});`

P2511	`q.emplace(nontype<&DB::connect>, std::move(db));`

Motivation

Not all user-defined objects have an `operator()`. To pass a user-defined object to a parameter of `std::move_only_function`, one must be able to use that object as

```
obj(some, args)
```

But often, the object is designed to be used as

```
obj.send(some, args)
```

I want to use such an object with `move_only_function` as if the type of the object aliased its `send` member function to `operator()`.

Analysis

Why don’t people make their classes *callable*?

Given sufficient tag types, any member functions can be expressed in one overload set, namely `operator()`. But we are not in that direction and are not working with constructors – a typical context where the entity to call is anonymous. Naming functions differently is the best way to disambiguate.

When there is no demand for disambiguation, people who design the class can still prefer naming their only member function in the interface “run,” “parse,” and even “apply,” since clearly,

```
repl.run(line);
```

delivers more information than

```
repl(line);
```

Can we do this with wrapping?

Lambda expression is an option. We can pass a closure object instead of the original object

```
pack.start([obj{std::move(obj)}] (auto... args) mutable
           {
               return obj.send(args...);
           });
```

Perfect-forwarding will need more boilerplate

```
pack.start([obj{std::move(obj)}]
           <class ...T>(T &&... args) mutable
           {
               return obj.send(std::forward<T>(args)...);
           });
```

And don’t forget that we are using `obj` only as an lvalue in this example. A lambda’s captures are unaware of the value category of the closure object unless using `std::forward_like` [1] together with an explicit object parameter.

Let’s just say that lambda expression is not suitable for expressing the intention of designating a differently named member function.

And it’s a part of the reason why we have `std::bind_front`. You can rewrite the example above as

```
pack.start(std::bind_front(&Cls::send, std::move(obj)));
```

But `bind_front` incurs a cost. Sometimes, the space penalty – size of a pointer to member function, is greater than the `obj` itself. Other than that, the codegen cost is also high.

Why the ask has to have something to do with type-erasure?

Let’s recall function pointers – the most basic form of type-erasures. You can assign different functions to the same function pointer:

```
typedef int cmp_t(const char *, const char *);
cmp_t *p = strcmp;
p = strcasecmp;
```

At first glance, here we did not erase type. The turth is that we erased “nontype” compile-time information – `&strcmp` and `&strcasecmp`. In a hypothetical language, every function could be of different types, like what the following piece of legal C++ code shows:

```
p = [](auto lhs, auto rhs)
    {
        return string_view(lhs).compare(rhs);
    };
```

There are two ways to understand this:

1. The function pointer erased type `T` from the closure.
2. C++ offshored the compile-time identity of functions into nontype values.

They are equivalent, as you may get those nontype values back into types anytime:

```
template<auto V> struct nontype_t {};
static_assert(!is_same_v<nontype_t<strcmp>, nontype_t<strcasecmp>>);
```

Type erasure is called “type erasure” because it makes code that should depend on different types, not dependent. It wouldn’t be surprising if the code supposed to be dependent were *value-dependent*. So it seems that a type-erased call wrapper is suitable for erasing the nontype information, `&Cls::send`, from the expression

```
obj.send(some, args)
```

to make the code that uses `obj` depends on neither `&Cls::send` nor `Cls`, where the latter case is what being erased if `obj` were *callable*.

Proposal

This proposal consists of two parts. First, it adds `nontype_t` and `nontype` to the `<utility>` header. They are similar to `in_place_type_t` and `in_place_type`, except each of the former two accepts a nontype template parameter with `auto` rather than type template parameters.

Second, it adds `nontype_t<V>` to `move_only_function`’s constructors to allow you to do the following things:

```
pack.start({nontype<&Cls::send>, std::move(obj)}); // 1
pack.start({nontype<&Cls::send>, &obj});           // 2
```

In the first case, the `move_only_function` parameter owns `obj`. In the second case, the parameter holds a reference to `obj`. But this does not mean that we will dereference the pointer in the second case. It works here because the *`INVOKE`* protocol calls a pointer-to-member on an object pointer. If `Cls::send` were an *explicit object member function*, case 2 would stop working.

Another constructor converts a single `nontype<V>` to `move_only_function`. It is merely a shortcut to initialize from a callable object if we can pass it using a nontype template parameter.

```
move_only_function<cmp_t> fn = strcmp;
fn = nontype<strcasecmp>;  // new
```

This revision proposes the one-argument and the two-argument `nontype_t` constructors for `std::function` as well. The similar change for `function_ref` [2] is handled in P2472[3].

Third, clone `move_only_function`’s `in_place_type_t<T>` constructors and prepend the `nontype_t<V>` parameters. This will give us two more constructors.

Discussion

How do other programming languages solve this problem?

Java® did not designate a magic method serving `operator()`'s role. Instead, any interface with a single abstract method is deemed a **functional interface**. When passing a lambda expression to a parameter of a functional interface, Java produces a closure object that implements this interface. So it doesn't matter what the method's name is; it may be `void accept(T)`, `R apply(T)`, etc. But you don't have to use a lambda if your `obj` doesn't implement the functional interface. Method references are a more straightforward way to produce a closure object. For example, `obj::consume` can make a closure object supporting the `accept` method.

Python designates `__call__` to be the magic method to make an object callable. If you want to call a different method to fulfill the `typing.Callable` requirement, you may pass a **bound method** like `obj.send`. A method in Python has a `__self__` attribute to store the class instance.

C♯, similar to Java, doesn't support overloading the call operator. However, its **delegate** language facility allows quickly defining a functional interface. Unlike Java, you cannot "implement" a delegate, so you must create delegate objects using a syntax similar to Python's bound methods.

In a parallel universe, C++ with C++0x Concepts provides **concept maps** as a general mechanism for adapting a de facto interface to be used in a component that expects a common, but different interface. Here, to enable type `Cls` for being used in `move_only_function`, we can specify a mapping using Richard's generalized alias declaration[4]:

```
template<class... Args>
concept_map invocable<Cls, Args...>
{
    using operator() = Cls::send;
};
```

To make this adaptation local to the users' code, they can define the `concept_map` in their own namespace.[5]

Why this proposal is different from delegates and other solutions?

The solution given by concept maps has the right model. But *invocable* is not only a concept. It is centralized in a programming paradigm. That might be why the other solutions widely used in practice allow forming the adaptations that are different from use to use.

The rest of the solutions, such as delegates, are all language features that work only with member functions.

However, in C++, until C++20, functional interface means `std::function`. There are also `packaged_task`, `folly::Function`, `llvm::function_ref` ... There is no generic functional interface that fits all needs.

We are proposing a framework for enabling designating a different `operator()` when initializing any functional interface in C++. A third-party library can also add `nontype_t<V>` to their type-erased call wrappers' constructor overload sets. The `V` they accept may be more permissive or restrictive, the storage policy they chose may be adaptive or pooled, but the end-users can enjoy expressing the same idea using the same syntax.

And a C++ class's interface consists not only of member functions. The NTTP callable, `V`, can be a pointer to explicit object member function – in other words, you can treat a free function as that member function. You can even rewrite that `obj`'s call operator with another structural object with an `operator()`:

```
nontype<[](Cls) { runtime_work(); }>
```

The proposed solution supports all these to satisfy users' expectations for C++.

Should we add those constructors to all type-erased call wrappers?

This revision proposes additions to `std::function`. Hope this does not create ABI concerns.

The typical uses of `packaged_task` do not seem to value codegen high enough to justify adding the `nontype_t<V>` constructors.

Should type-passing call wrappers support NTTP callables?

Strictly speaking, they are outside the scope of this paper. But some type-passing call wrappers that require factory functions have an attractive syntax when accepting NTTP callables – you can pass them in function templates' *template-argument-list*. So let me break down the typical ones: `bind_front<V>`, `not_fn<V>()`, and `mem_fn<V>()`.

Supporting `bind_front<&Cls::send>(obj)` eliminates the space penalty of `bind_front(&Cls::send, obj)`. But, please be aware that if `bind_front` appears alone, the compiler has no pressure optimizing type-passing code. Hence, the new form only makes sense if a type-erasure later erases the underlying wrapper object. But a type-erased call wrapper already requires wrapping the target object. This double-wrapping downgrades the usability of those call wrappers:

1. One cannot in-place constructs `obj` in an owning call wrapper bypassing `bind_front`;
2. One must write `bind_front<&Cls::send>(std::ref(obj))` to archive reference semantics even if this expression is about to initialize a `function_ref`, defeating half of the purpose of `function_ref`.

The need of type-erasing a predicate such as `not_fn(p)` seems rare. STL algorithms that take predicates have a type-passing interface.

The uses of `std::mem_fn` largely diminished after introducing `std::invoke`.

Should `nontype_t` itself be callable?

Rather than overloading `mem_fn` to take NTTP callables, adding an `operator()` to `nontype_t` will have the same, or arguably better, effect:

```
std::transform(begin(v), end(v), it, nontype<&Cls::pmd>);
```

And doing so can make `bind_front(nontype<&Cls::send>, obj)` automatically benefit from better codegen in a quality implementation of `std::bind_front`. However, this raises both API and ABI concerns.

In terms of API, `nontype_t`'s meaning should be entirely up to the wrapper. I don't see a problem if a call wrapper interprets

```
auto fn = C{fp, nontype<std::fputs>};
```

as binding `fp` to the last parameter.

When it comes to ABI, the return type of `bind_front` is opaque but not erased – it can be at the ABI boundary. So if a type-passing wrapper like `bind_front` later wants to handle `nontype_t` differently as a QoI improvement, it breaks ABI.

Can some form of lambda solve the problem?

The previous discussion (1, 2) revealed how large the design space is and how the problem ties to the library. This section will use a specific paper in the past as an example to show how these factors can affect language design.

There have been discussions about whether C++ should have expression lambda[6]

```
priority_queue pq([][&1.id() < &2.id()], input);
```

to make lambda terse. Expression lambdas aim to address the difficulty of introducing parameters. But in our motivating example, we forwarded all parameters without introducing any. So expression lambda doesn't directly respond to the problem.

So the ask will need to expand the scope of *lambda* to "anything that can produce an anonymous closure object." It is reasonable as other languages have similar sugars. For example, Java's method references and lambda expressions share VM mechanisms.[7]

In that case, let's prototype the ask: Why don't we write the following and make everything work?

```
q.push_back(db.connect);
```

Recall that `q` is a container of `std::move_only_function` from Introduction, so the first question will be what `db.connect` means. Luckily, Andrew Sutton had an answer to that in 2016.[8] Here is an example from his paper:

```
struct S
{
  void f(int&);
  void f(std::string&);
};

S s;
std::transform(first, last, s.f);
```

`s.f` produces (applied minor corrections):

```
[&s] <class... Args>(Args&&... args)
  -> decltype(s.f(std::forward<Args>(args)...))
{
  return s.f(std::forward<Args>(args)...);
}
```

The above suggests that, in our example, `db.connect` captures `db` by reference.

But `move_only_function` is supposed have a unique copy of `db`! In the motivating example, we `std::move(db)` into an element of `q`. So maybe `s` should mean “capture by value” in `s.f`, and we write `std::move(db).connect`?

Assume it is the case. What happens if there is another function `retry` taking `function_ref`:

```
retry(db.connect);  // a few times
```

Given the modified semantics, the above should mean “capture `db` by making a copy, and pass the closure by reference using `function_ref`.” Which is, of course, not satisfying. `std::ref(db)` won’t help this time, so let’s go with

```
retry((&db)->connect);  // a few times
```

Now `db.connect` and `(&db)->connect` have different meanings. This implies that if we had a pointer to `db`,

```
auto p = &db;
```

`(*p).connect` and `p->connect` will have different meanings. This goes against the common expectation on C++ ([over.call.func]/2 (https://eel.is/c++draft/over.call.func#2)):

> [...] the construct `A->B` is generally equivalent to `(*A).B`

Let’s take another angle. Instead of moving `db`, what if we want to construct an object of `DB` in the new element in place?

It’s simple using a `nontype` constructor. We only need to go from

```
q.emplace(nontype<&DB::connect>, std::move(db));
```

to

```
q.emplace(nontype<&DB::connect>,
          std::in_place_type<DB>, "example.db", 100ms, true);
```

I don't know of a solution that involves capturing. `DB("example.db", 100ms, true).connect` will result in moving a subobject along with the closure. And more importantly, it requires `DB` to be movable, which adds more to `move_only_function`'s minimal requirements.

It seems that C++ lambdas are not only verbose to introduce parameters but also tricky to capture variables. A solution that couples naming a different *`id-expression`* after `.` operator with captures will face problems when working with varying type-erased call wrappers.

But if expression lambda solves the problem of introducing parameters, can we replace the problems caused by capturing variables with the problem we solved?

```
q.emplace(nontype<[][&1::connect]>, std::move(db));
```

This WORKS. A captureless lambda is of a structral type. It solves the problem of selecting a particular overload or specialization when `connect` is an overload set. In Andrew's example,

```
struct S
{
  void f(int&);
  void f(std::string&);
};
```

Right now, to select the first overload, we have to write `nontype<(void (S::*)(int&))&S::f>`; with an expression lambda, it's as simple as `nontype<[][&1.f]>`.

As long as we decoupled naming from wrapping, a language design can relieve itself from making every library happy with a single type-passing language feature that does wrapping for you. In that situation, the library additions and the language additions can evolve in parallel and work together in the end.

Can this proposal target post-C++23?

Suppose all implementations of `std::move_only_function` are prepared for this to come, yes. A naïve implementation that uses vtable can later add value-dependent subtypes without breaking ABI; others need to be careful.

On the other hand, the `nontype_t<V>` constructors allow the users to bind implicit or explicit object member functions with the same cost. So it is reasonable to ship them in the same standard that ships `std::move_only_function` and "deducing this."[9]

Prior Art

The earliest attempt to bind an object with a member function I can find is Rich Hickey's "Callbacks in C++ Using Template Functors"[10] back in 1994.

Borland C++ has a language extension – the `__closure` keyword.[11] It is very similar to the hypothetical `__bound` keyword in Rich's article. However, you may not `delete` a pointer to closure or initialize it with a function.

This proposal is inspired by an earlier revision of P2472[3:1].

Wording

The wording is relative to N4901.

Part 1.

Add new templates to [utility.syn] (https://eel.is/c++draft/utility.syn), header `<utility>` synopsis:

```
namespace std {
  [...]

  template<size_t I>
    struct in_place_index_t {
      explicit in_place_index_t() = default;
    };

  template<size_t I> inline constexpr in_place_index_t<I> in_place_index{};

  // nontype argument tag
  template<auto V>
    struct nontype_t {
      explicit nontype_t() = default;
    };

  template<auto V> inline constexpr nontype_t<V> nontype{};
}
```

Revise definitions in [func.def] (https://eel.is/c++draft/func.def):

> […]
>
> A *target object* is the ~~callable~~ object held by a call wrapper for the purpose of calling.
>
> A call wrapper type may additionally hold a sequence of objects and references that may be passed as arguments to the call expressions involving the target object. […]

Part 2.

Add new signatures to [func.wrap.func.general] (https://eel.is/c++draft/func.wrap.func.general) synopsis:

> […]

```
template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
public:
  using result_type = R;

  // [func.wrap.func.con], construct/copy/destroy
  function() noexcept;
  function(nullptr_t) noexcept;
  template<auto f> function(nontype_t<f>) noexcept;
  function(const function&);
  function(function&&) noexcept;
  template<class F> function(F&&);
  template<auto f, class T> function(nontype_t<f>, T&&);
```

> […]

Insert the following to [func.wrap.func.general] (https://eel.is/c++draft/func.wrap.func.general) after paragraph 3:

> The `function` class template is a call wrapper [func.def] (https://eel.is/c++draft/func.def) whose call signature [func.def] (https://eel.is/c++draft/func.def) is `R(ArgTypes...)`.
>
> Within this subclause, *`call-args`* is an argument pack with elements that have types `ArgTypes&&...` respectively.

Modify [func.wrap.func.con] (https://eel.is/c++draft/func.wrap.func#con) as indicated:

> […]

```
function(nullptr_t) noexcept;
```

Postconditions: `!*this`.

`template<auto f> function(nontype_t<f>) noexcept;`

Constraints: `is_invocable_r_v<R, decltype(f), ArgTypes...>` is `true`.

Postconditions: `*this` has a target object. Such an object and `f` are template-argument-equivalent [temp.type] (https://eel.is/c++draft/temp.type).

Remarks: The stored target object leaves no type identification [expr.typeid] (https://eel.is/c++draft/expr.typeid) in `*this`.

`template<class F> function(F&& f);`

Let `FD` be `decay_t<F>`.

Constraints:

- `is_same_v<remove_cvref_t<F>, function>` is `false`, and
- `FD` is Lvalue-Callable [func.wrap.func] (https://eel.is/c++draft/func.wrap.func) for argument types `ArgTypes...` and return type `R`.

Mandates:

- `is_copy_constructible_v<FD>` is `true`, and
- `is_constructible_v<FD, F>` is `true`.

Preconditions: `FD` meets the *Cpp17CopyConstructible* requirements.

Postconditions: `!*this` is `true` if any of the following hold:

- `f` is a null function pointer value.
- `f` is a null member pointer value.
- `remove_cvref_t<F>` is a specialization of the `function` class template, and `!f` is `true`.

Otherwise, `*this` has a target object of type `FD` direct-non-list-initialized with `std::forward<F>(f)`.

Throws: Nothing if `FD` is a specialization of `reference_wrapper` or a function pointer type. Otherwise, may throw `bad_alloc` or any exception thrown by the initialization of the target object.

Recommended practice: Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where `f` refers to an object holding only a pointer or reference to an object and a member function pointer.

```
template<auto f, class T> function(nontype_t<f>, T&& x);
```

> Let `D` be `decay_t<T>`.
>
> *Constraints*: `is_invocable_r_v<R, decltype(f), D&, ArgTypes...>` is `true`.
>
> *Mandates*:
>
> - `is_copy_constructible_v<D>` is `true`, and
> - `is_constructible_v<D, T>` is `true`.
>
> *Preconditions*: `D` meets the *Cpp17CopyConstructible* requirements.
>
> *Postconditions*: `*this` has a target object `d` of type `D` direct-non-list-initialized with `std::forward<T>(x)`. `d` is hypothetically usable in a call expression, where `d(`*`call-args...`*`)` is expression equivalent to `invoke(f, d,` *`call-args...`*`)`
>
> *Throws*: Nothing if `D` is a specialization of `reference_wrapper` or a pointer type. Otherwise, may throw `bad_alloc` or any exception thrown by the initialization of the target object.
>
> *Recommended practice*: Implementations should avoid the use of dynamically allocated memory for small callable objects, for example, where `f` refers to an object holding only a pointer or reference to an object.
>
> [...]

```
R operator()(ArgTypes... args) const;
```

> *Returns*: *`INVOKE`*`<R>(f, std::forward<ArgTypes>(args)...)` [func.require] (https://eel.is/c++draft/func.require), where `f` is the target object [func.def] (https://eel.is/c++draft/func.def) of `*this`.
>
> *Throws*: `bad_function_call` if `!*this`; otherwise, any exception thrown by the target object.

Modify [func.wrap.func.targ] (https://eel.is/c++draft/func.wrap.func#targ) as indicated:

```
const type_info& target_type() const noexcept;
```

> *Returns*: If `*this` has a target of type `T` and the target did not waive its type identification in `*this`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<class T>       T* target() noexcept;
template<class T> const T* target() const noexcept;
```

> *Returns*: If `target_type() == typeid(T)` a pointer to the stored function target; otherwise a null pointer.

Part 3.

Add new signatures to [func.wrap.move.class] (https://eel.is/c++draft/func.wrap.move.class) synopsis:

> […]

```
  template<class R, class... ArgTypes>
  class move_only_function<R(ArgTypes...) cv ref noexcept(noex)> {
  public:
    using result_type = R;

    // [func.wrap.move.ctor], constructors, assignment, and destructor
    move_only_function() noexcept;
    move_only_function(nullptr_t) noexcept;
    move_only_function(move_only_function&&) noexcept;
    template<auto f> move_only_function(nontype_t<f>) noexcept;
    template<class F> move_only_function(F&&);
    template<auto f, class T> move_only_function(nontype_t<f>, T&&);
    template<class T, class... Args>
      explicit move_only_function(in_place_type_t<T>, Args&&...);
    template<auto f, class T, class... Args>
      explicit move_only_function(
        nontype_t<f>,
        in_place_type_t<T>,
        Args&&...);
    template<class T, class U, class... Args>
      explicit move_only_function(in_place_type_t<T>, initializer_list<U>, Args&&.
    template<auto f, class T, class U, class... Args>
      explicit move_only_function(
        nontype_t<f>,
        in_place_type_t<T>,
        initializer_list<U>,
        Args&&...);

    move_only_function& operator=(move_only_function&&);
    move_only_function& operator=(nullptr_t) noexcept;
    template<class F> move_only_function& operator=(F&&);

    ~move_only_function();

    // [func.wrap.move.inv], invocation
    explicit operator bool() const noexcept;
    R operator()(ArgTypes...) cv ref noexcept(noex);

    // [func.wrap.move.util], utility
    void swap(move_only_function&) noexcept;
    friend void swap(move_only_function&, move_only_function&) noexcept;
    friend bool operator==(const move_only_function&, nullptr_t) noexcept;

  private:
    template<class... T>
      static constexpr bool is-invocable-using = see below;     // exposition only
    template<class VT>
      static constexpr bool is-callable-from = see below;       // exposition only
    template<auto f, class T>
```

```
    static constexpr bool is-callable-as-if-from = see below; // exposition only
  };
}
```

Insert the following to [func.wrap.move.class] (https://eel.is/c++draft/func.wrap.move.class) after paragraph 1:

> [...] These wrappers can store, move, and call arbitrary callable objects, given a call signature.
>
> Within this subclause, *call-args* is an argument pack with elements that have types `ArgTypes&&...` respectively.

Modify [func.wrap.move.ctor] (https://eel.is/c++draft/func.wrap.move.ctor) as indicated:

```
template<class... T>
  static constexpr bool is-invocable-using = see below;
```

> If *noex* is true, *is-invocable-using*`<T...>` is equal to:
>
> `is_nothrow_invocable_r_v<R, T..., ArgTypes...>`
>
> Otherwise, *is-invocable-using*`<T...>` is equal to:
>
> `is_invocable_r_v<R, T..., ArgTypes...>`

```
template<class VT>
  static constexpr bool is-callable-from = see below;
```

> ~~If *noex* is true, *is-callable-from*`<VT>` is equal to:~~
>
> ~~– `is_nothrow_invocable_r_v<R, VT cv ref, ArgTypes…> &&`~~
> ~~– `is_nothrow_invocable_r_v<R, VT inv-quals, ArgTypes…>`~~
>
> ~~Otherwise,~~ *is-callable-from*`<VT>` is equal to:
>
> *is-invocable-using*`<VT cv ref> &&`
> *is-invocable-using*`<VT inv-quals>`
> ~~– `is_invocable_r_v<R, VT cv ref, ArgTypes…> &&`~~
> ~~– `is_invocable_r_v<R, VT inv-quals, ArgTypes…>`~~

```
template<auto f, class T>
  static constexpr bool is-callable-as-if-from = see below;
```

> *is-callable-as-if-from*<f, VT> is equal to:
>
> ```
>  is-invocable-using<decltype(f), VT inv-quals>
> ```
>
> [...]

```
template<auto f> move_only_function(nontype_t<f>) noexcept;
```

> *Constraints*: `is-invocable-using<decltype(f)>` is `true`.
>
> *Postconditions*: `*this` has a target object. Such an object and `f` are template-argument-equivalent [temp.type] (https://eel.is/c++draft/temp.type).

```
template<class F> move_only_function(F&& f);
```

Let `VT` be `decay_t<F>`.

Constraints:

- `remove_cvref_t<F>` is not the same type as `move_only_function`, and
- `remove_cvref_t<F>` is not a specialization of `in_place_type_t`, and
- *`is-callable-from`*`<F>` is `true`.

Mandates: `is_constructible_v<VT, F>` is `true`.

Preconditions: `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

Postconditions: `*this` has no target object if any of the following hold:

- `f` is a null function pointer value, or
- `f` is a null member pointer value, or
- `remove_cvref_t<F>` is a specialization of the `move_only_function` class template, and `f` has no target object.

Otherwise, `*this` has a target object of type `VT` direct-non-list-initialized with `std::forward<F>(f)`.

Throws: Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a function pointer or a specialization of `reference_wrapper`.

```
template<auto f, class T> move_only_function(nontype_t<f>, T&& x);
```

Let `VT` be `decay_t<T>`.

Constraints: *`is-callable-as-if-from`*`<f, VT>` is `true`.

Mandates: `is_constructible_v<VT, T>` is `true`.

Preconditions: `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

Postconditions: `*this` has a target object `d` of type `VT` direct-non-list-initialized with `std::forward<T>(x)`. `d` is hypothetically usable in a call expression, where `d(`*`call-args...`*`)` is expression equivalent to `invoke(f, d,` *`call-args...`*`)`

Throws: Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a pointer or a specialization of `reference_wrapper`.

```
template<class T, class... Args>
  explicit move_only_function(in_place_type_t<T>, Args&&... args);
template<auto f, class T, class... Args>
  explicit move_only_function(
    nontype_t<f>,
    in_place_type_t<T>,
    Args&&... args);
```

Let `VT` be `decay_t<T>`.

Constraints:

- `is_constructible_v<VT, ArgTypes...>` is `true`, and
- *`is-callable-from`*`<VT>` is `true` for the first form or *`is-callable-as-if-from`*`<f, VT>` is `true` for the second form.

Mandates: `VT` is the same type as `T`.

Preconditions: `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

Postconditions: `*this` has a target object `d` of type `VT` direct-non-list-initialized with `std::forward<Args>(args)...`. With the second form, `d` is hypothetically usable in a call expression, where `d(`*`call-args…`*`)` is expression equivalent to `invoke(f, d,` *`call-args…`*`)`

Throws: Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a ~~function~~ pointer or a specialization of `reference_wrapper`.

```
template<class T, class U, class... Args>
  explicit move_only_function(in_place_type_t<T>, initializer_list<U> ilist, Args&
template<auto f, class T, class U, class... Args>
  explicit move_only_function(
    nontype_t<f>,
    in_place_type_t<T>,
    initializer_list<U> ilist,
    Args&&... args);
```

Let `VT` be `decay_t<T>`.

Constraints:

- `is_constructible_v<VT, initializer_list<U>&, ArgTypes...>` is `true`, and
- *`is-callable-from`*`<VT>` is `true` for the first form or *`is-callable-as-if-from`*`<f, VT>` is `true` for the second form.

Mandates: `VT` is the same type as `T`.

Preconditions: `VT` meets the *Cpp17Destructible* requirements, and if `is_move_constructible_v<VT>` is `true`, `VT` meets the *Cpp17MoveConstructible* requirements.

Postconditions: `*this` has a target object `d` of type `VT` direct-non-list-initialized with `ilist, std::forward<Args>(args)...`. With the second form, `d` is hypothetically usable in a call expression, where `d(`*`call-args…`*`)` is expression equivalent to `invoke(f, d,` *`call-args…`*`)`

Throws: Any exception thrown by the initialization of the target object. May throw `bad_alloc` unless `VT` is a ~~function~~ pointer or a specialization of `reference_wrapper`.

[...]

```
R operator()(ArgTypes... args) cv ref noexcept(noex);
```

Preconditions: `*this` has a target object.

Effects: Equivalent to:

```
return INVOKE<R>(static_cast<F inv-quals>(f), std::forward<ArgTypes>
(args)...);
```

where `f` is an lvalue designating the target object of `*this` and `F` is the type of `f`.

Implementation Experience

This project (https://github.com/zhihaoy/nontype_functional) implements `std::function` and `function_ref` with the `nontype_t<V>` constructors. An implementation for `move_only_function` is on the way.

Acknowledgments

Thank Ryan McDougall and Tomasz Kamiński for providing valuable feedback to the paper.

References

1. Ažman, Gašper. P2445R0 *std::forward_like*. http://wg21.link/p2445r0 (http://wg21.link/p2445r0) ↩

2. Romeo, et al. P0792R8 *function_ref: a type-erased callable reference*. http://wg21.link/p0792r8 (http://wg21.link/p0792r8) ↩

3. Waterloo, J.J. P2472R1 *make function_ref more functional*. http://wg21.link/p2472r1 (http://wg21.link/p2472r1) ↩ ↩

4. Smith, Richard. P0945R0 *Generalizing alias declarations*. http://wg21.link/p0945r0 (http://wg21.link/p0945r0) ↩

5. Siek, Jeremy. N2098 *Scoped Concept Maps*. http://wg21.link/n2098 (http://wg21.link/n2098) ↩

6. Revzin, Barry. *Why were abbrev. lambdas rejected?* https://brevzin.github.io/c++/2020/01/15/abbrev-lambdas/ (https://brevzin.github.io/c++/2020/01/15/abbrev-lambdas/) ↩

7. Goetz, Brian. *Translation of Lambda Expressions .* https://cr.openjdk.java.net/~briangoetz/lambda/lambda-translation.html (https://cr.openjdk.java.net/~briangoetz/lambda/lambda-translation.html) ↩

8. Sutton, Andrew. P0119R2 *Overload sets as function arguments.* http://wg21.link/p0119r2 (http://wg21.link/p0119r2) ↩

9. Ažman et al. P0847R7 *Deducing this*. http://wg21.link/p0847r7 (http://wg21.link/p0847r7) ↩

10. Hickey, Rich. *Callbacks in C++ Using Template Functors*. http://www.tutok.sk/fastgl/callback.html (http://www.tutok.sk/fastgl/callback.html) ↩

11. __closure. *Language Support for the RAD Studio Libraries (C++)*. https://docwiki.embarcadero.com/RADStudio/Sydney/en/Closure (https://docwiki.embarcadero.com/RADStudio/Sydney/en/Closure) ↩